

SkillSync: Explainable, Efficient, and Transparent Resume Evaluation

Maximilian Schulten¹ Jack Alberto¹ Tommaso Beretta¹ William Hugus¹ Nathaniel Hahn¹ Dr. Shaui Xu¹

¹Case Western Reserve University School of Engineering, Department of Computer and Data Sciences



Abstract

Job searching is often a time consuming and inefficient process that relies heavily on user judgement to assess their own fit for each job opening. With the rise of online job boards that everyone can access, the online job market has become overly competitive in recent years. This project, SkillSync, develops a lightweight, privacy focused, and data driven system for matching resumes to job descriptions.

The goal of this project is to automate and improve the user experience of job searching by generating a list of job openings for which the user is most qualified for. The system operates through a pipeline that includes skill extraction from the resume, job data aggregation, matching process, and generating the list of top matches.

System Outline

We divide **SkillSync** into four main steps:

- Skill Extraction** The user uploads a resume and the skills are extracted while personal identifiable information is removed.
- Job Aggregation** We query multiple APIs to maximize job opportunities; each job has key features extracted.
- Skill Matching** The skills extracted during step on are matched with the job features extracted in step two. Keyword search and a matching model are applied.
- Match Presentation** We present a the ordered top matches to the user in a clean and functional UI.

Skill Extraction and Preprocessing

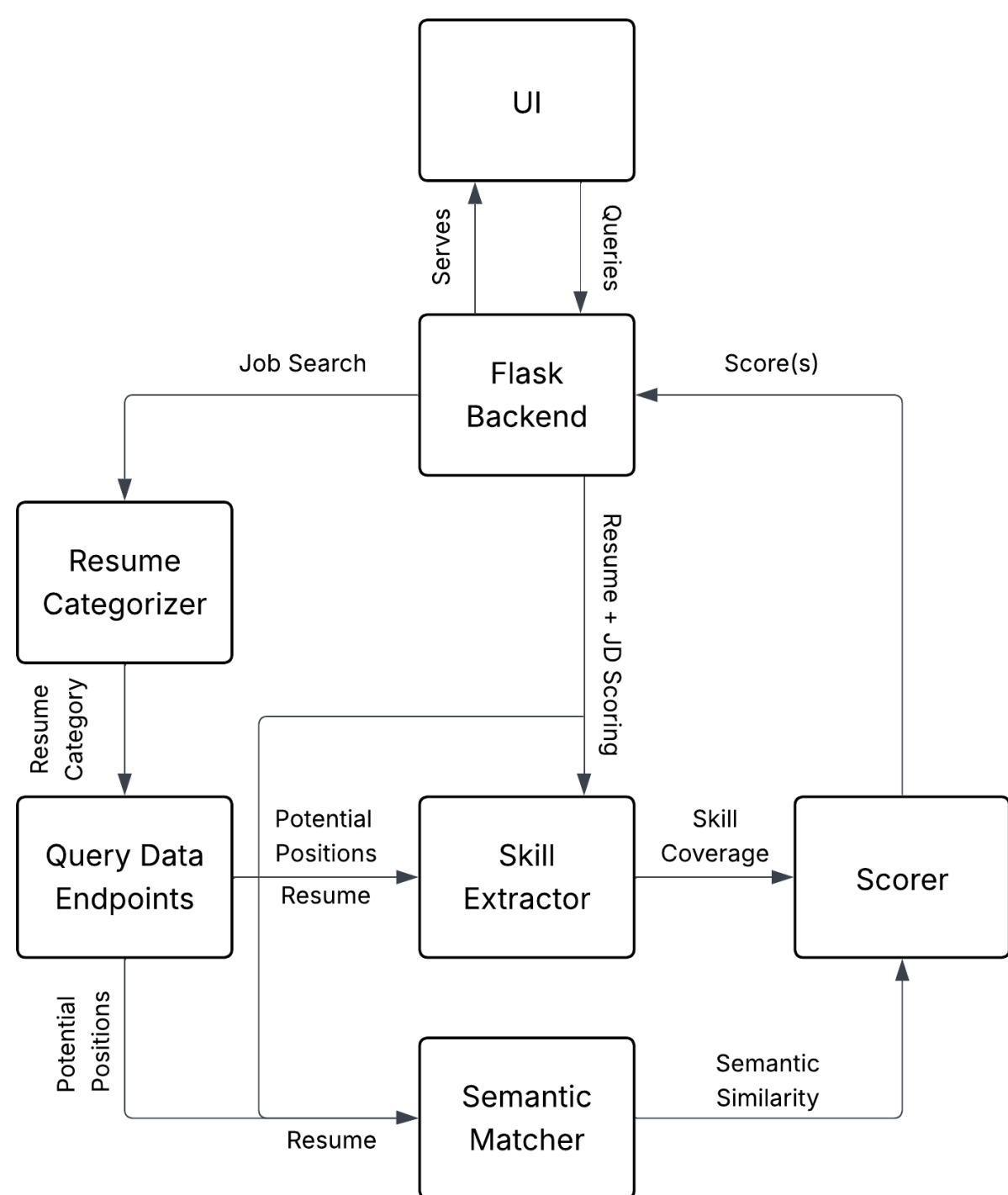
The skill extraction modules contain the data cleaning, resume classifier and skill extraction, the result of this module is a full understanding of the users resume without and personally identifying information (PII).

- Data Cleaning** Our pipeline strips non alphanumeric characters and uncommon punctuation along with extra whitespace. The expectation is that stripping whitespace only removes structural information and does not limit our ability to extract skills. A combination of regular expressions and a named entity recognition model was used to recognize and obfuscate PII such as names, locations, emails, organizations, and phone numbers. We understand implicit bias within trained models and chose to remove PII rather than substitute psuedonyms.
- Resume Classification** To reduce the search space, we use the resume industry as a heuristic for matching. We employ a simple linear support vector machine classifier (SVM) that classifies resumes into large industry buckets. Resumes are then pre-processed using an embedding transformer model to create vectors that our model could predict on. We mapped the resume categories to a broader 12 category set and then trained our classifier, resulting in a **>90%** top-2 accuracy.
- Skill Extraction** We chose a hybrid approach using spaCy PhraseMatcher, a fast dictionary-based skill matching against 99k+ canonical skill terms and GLiNER (Generalist NER Model), a bidirectional transformer. The PhraseMatcher allowed for rapid keyword matching while, GLiNER allowed for semantic information to be extracted like education differences (BS, MS, PHD) This was a difficult process due to some candidates providing enumerated lists of skills, while others chose to let their experience represent their skills.

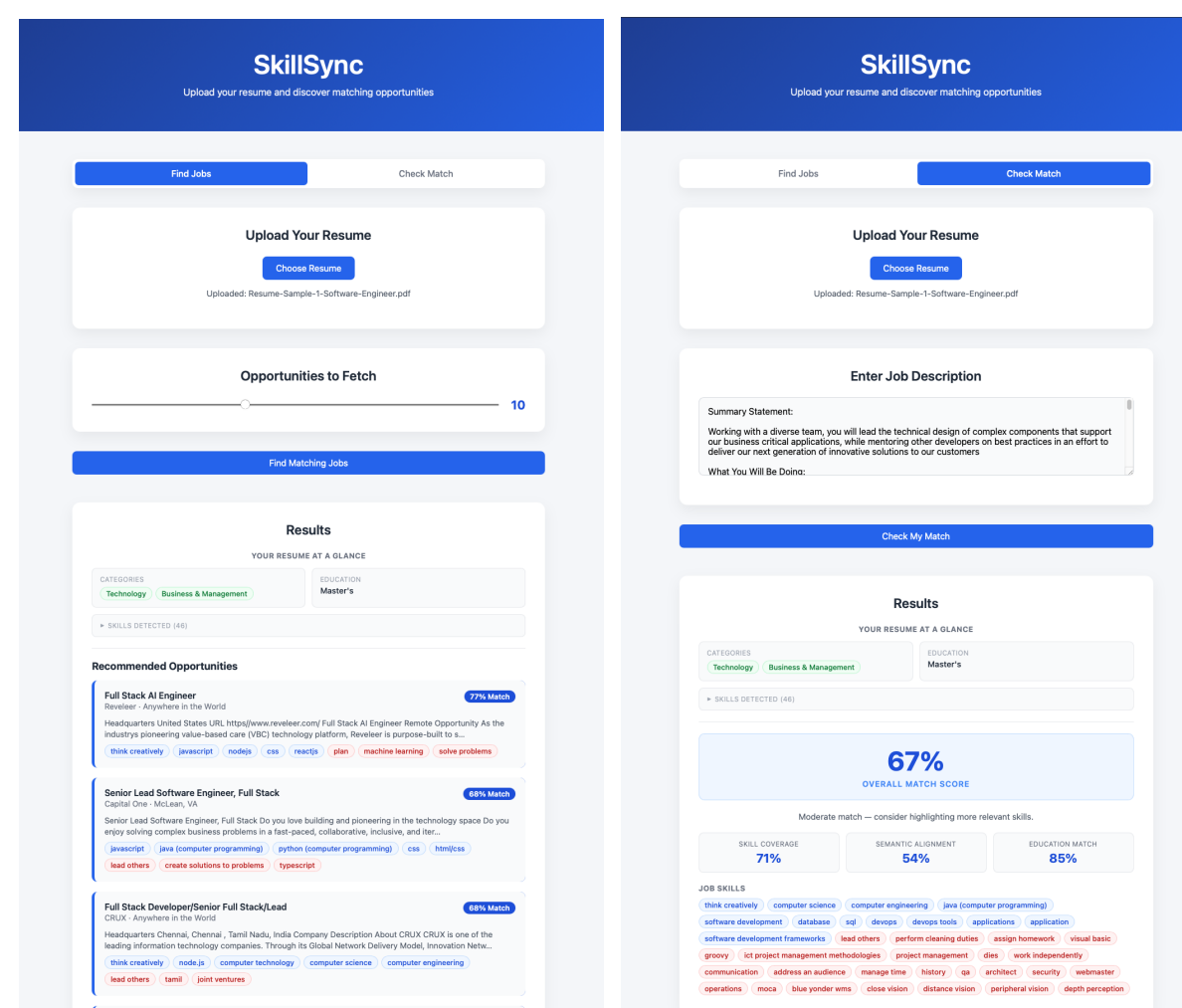
System Architecture

SkillSync is built as a three-tier web application. The frontend is a single-page interface where users paste their resume and receive a ranked list of job matches. It communicates with a Flask backend that exposes two REST endpoints, /find-jobs for full pipeline execution and /score-job for scoring a single listing. The backend orchestrates a multi-stage NLP pipeline: resume parsing and classification, job aggregation, skill extraction, and composite scoring. All components are containerized via Docker with memory limits set to the resource demands of the embedded transformer models.

System Architecture and Examples



System Architecture



Front End Examples

Data Aggregation

Jobs are fetched asynchronously from eight external job boards: Jobcity, RemoteOK, The Muse, Himalayas, We Work Remotely, Findwork.dev, USAJobs, and Adzuna. These job boards provide coverage over the majority of the market including: remote, government, and general listings. Each source returns listings in its own schema; We normalize these into a unified Job object. Duplicates are removed by (title, company, source) key before scoring. Skill extraction draws on a canonical skill map of 99,000+ terms alongside a GLiNER transformer model, giving the system broad vocabulary coverage without relying on any single job board's taxonomy

Front End

The interface is minimal and clean, featuring a resume text input and a results panel. After submission, users see each job ranked by composite match score, with per-job breakdowns of matched skills, unmatched skill gaps, and sub-scores for skill coverage, semantic similarity, and education alignment. This transparency lets users understand why a job ranked highly rather than treating the score as a black box.

Matching Model

SkillSync scores each job listing against a parsed resume using a convex combination of three complementary signals:

$$S_{\text{total}} = \alpha \cdot S_{\text{skill}} + \beta \cdot S_{\text{sem}} + \gamma \cdot S_{\text{edu}}, \quad \alpha + \beta + \gamma = 1 \quad (1)$$

Default weights: $\alpha = \beta = 0.4, \gamma = 0.2$.

- Skill Coverage (S_{skill})**
Uses **all-MiniLM-L6-v2** embeddings and cosine similarity:

$$S_{\text{skill}} = \frac{1}{|J|} \sum_{j \in J} w \left(\max_{r \in R} \cos(e_j, e_r) \right) \quad (2)$$

where $w(x)$ rewards matches above threshold $\tau = 0.6$.

- Semantic Similarity (S_{sem})**
To capture contextual alignment beyond discrete skills, the resume and job description are each segmented into overlapping windows of 30 tokens with stride 10. The mean of per-window maximum cosine similarities $\bar{s}$ is mapped to a calibrated score via a truncated normal CDF fit empirically to 200 job-resume pairs ($\mu = 0.345, \sigma = 0.067$):

$$S_{\text{sem}} = \Phi_{\mu, \sigma}(\bar{s}), \quad \bar{s} = \frac{1}{|W_J|} \sum_{w \in W_J} \max_{v \in W_R} \cos(e_w, e_v) \quad (3)$$

- Education Coverage (S_{edu})**
Combines degree-level alignment and major similarity:

$$S_{\text{edu}} = 0.7 \cdot S_{\text{deg}} + 0.3 \cdot S_{\text{maj}} \quad (4)$$

S_{deg} is a hierarchical score (1.0 if the candidate meets or exceeds the required degree level, 0.5 if one level below, 0.0 otherwise). S_{maj} is the cosine similarity between embedded major strings, thresholded at $\tau_{\text{edu}} = 0.6$

Testing and Results

Our project features a full pytest suite meant to ensure consistent development among a multi-person team and deployment verification. These tests provide coverage across our entire build, preprocess, extract and inference modules allowing us to verify changes and catch bugs.

An example of our results are seen the in the front end demo, along with the figure below, which describes a test between different resumes against a business job description.

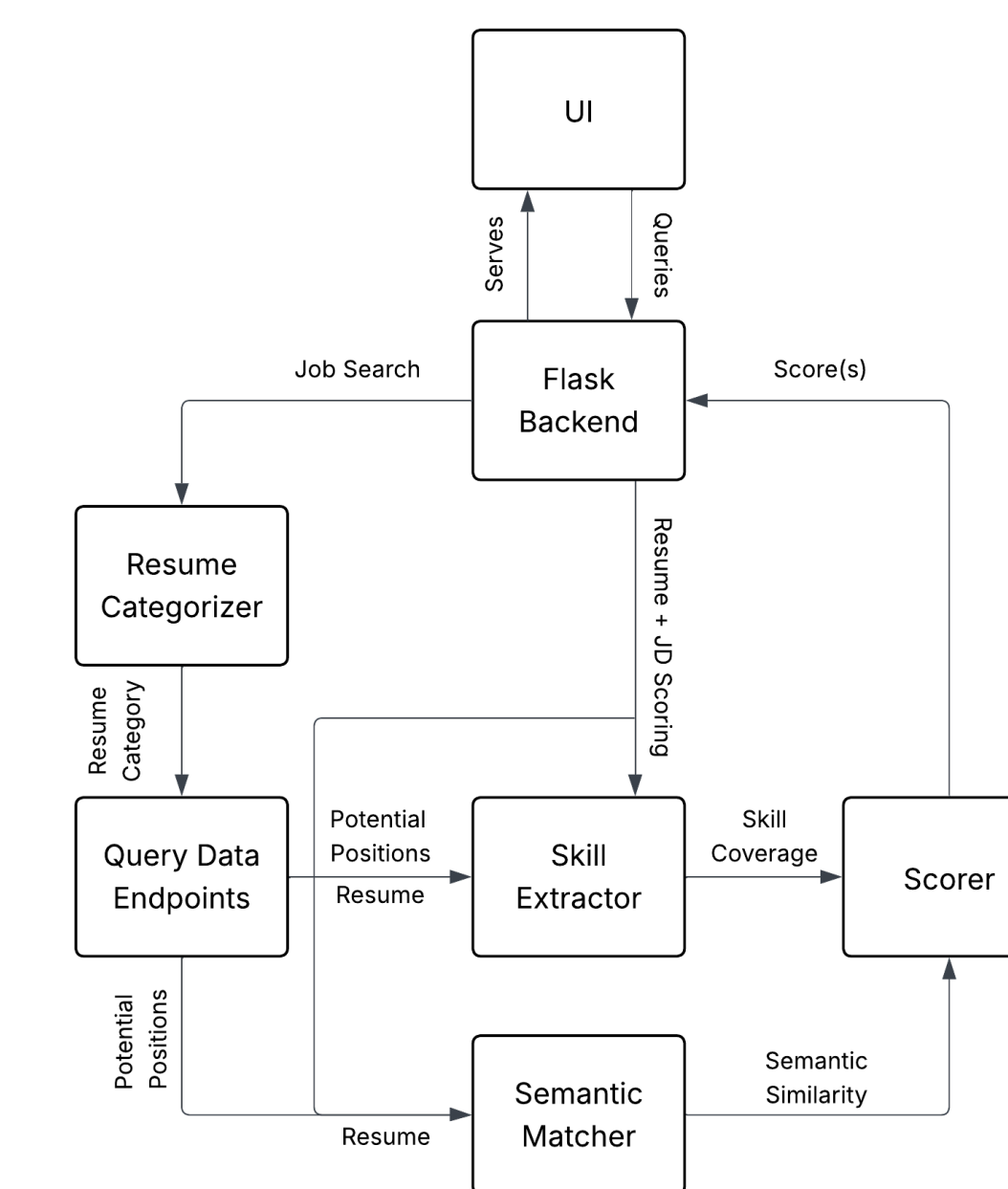


Figure 1. place holder